

MODÜL 06

CNN — Evrimsel Sinir Ağları

Convolutional Neural Networks

Konvolüsyon · Pooling · ResNet · EfficientNet · Feature Maps · Grad-CAM

Seviye	Orta-İleri
Kapsam	Konvolüsyon · Pooling · Mimari Evrimi · ResNet · Grad-CAM
Framework	NumPy · TensorFlow/Keras · PyTorch
Önceki Modül	Modül 05 — Regularization
Sonraki Modül	Modül 07 — Transfer Learning

1. Öğrenme Hedefleri

- ✓ Konvolüsyon işlemini matematiksel olarak tanımlamak ve padding/stride/dilation hesaplamak
- ✓ Çıkış boyutu formülünü doğru uygulamak: $H_{out} = \text{floor}((H_{in} + 2P - K) / S) + 1$
- ✓ Max pooling ve average pooling farkını açıklamak
- ✓ LeNet'ten EfficientNet'e CNN mimari evrimini özetlemek
- ✓ ResNet Skip Connection'ın vanishing gradient'i nasıl çözdüğünü açıklamak
- ✓ Feature map görselleştirmesiyle CNN'nin ne öğrendiğini yorumlamak
- ✓ Grad-CAM ile modelin odak bölgesini görselleştirmek
- ✓ TF/Keras ve PyTorch'ta CNN kodlamak

2. Konvolüsyon İşlemi

Konvolüsyon, bir filtrenin (kernel) giriş görüntüsü üzerinde kaydırılarak element-wise çarpım ve toplam yapmasıdır. Tam bağlantılı katmanlardan farkı: ağırlık paylaşımı (weight sharing) ve yerel bağlantı (local connectivity). Bu sayede parametre sayısı dramatik ölçüde azalır.

$$(I * K)[i, j] = \sum_m \sum_n I[i+m, j+n] \cdot K[m, n]$$

2.1 Çıkış Boyutu Formülü

$$H_{out} = \text{floor}((H_{in} + 2P - K) / S) + 1$$

H_in	K (Kernel)	P (Padding)	S (Stride)	H_out	Açıklama
8	3	0	1	6	Valid (no padding)
8	3	1	1	8	Same padding — boyut korunur
8	5	2	1	8	Same (5×5 filtre)
8	3	0	2	3	Stride=2 — boyut yarıya iner
32	3	1	1	32	Tipik CNN bloğu
32	3	1	2	16	Max pool alternatifi

2.2 Padding Türleri

Tür	Açıklama	Kullanım
Valid	Padding yok, çıkış küçülür	Boyut azaltma istendiğinde
Same	Çıkış = giriş boyutu (stride=1 ile)	Boyutu korumak için
Dilated	Kernel elemanları arasında boşluk	Büyük reseptif alan, az parametre

2.3 Konvolüsyon Filtreleri

- Kenar algılama (yatay): $K = \begin{bmatrix} -1, -1, -1 \\ 0, 0, 0 \\ 1, 1, 1 \end{bmatrix}$

- Kenar algılama (dikey): $K = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$
- Sobel: Yatay+Dikey kenar kombinasyonu
- Bulanıklaştırma: $K = (1/9) \cdot \text{ones}(3,3)$ (ortalama filtre)
- 1×1 Konvolüsyon: Kanal sayısını değiştir, boyutu koru (GoogLeNet)

3. Pooling Katmanları

Tür	İşlem	Boyut Etkisi	Kullanım
Max Pooling	Penceredeki maksimum değer	Küçülür ($2 \times 2 \rightarrow 1$)	CNN standart
Average Pooling	Penceredeki ortalama değer	Küçülür	GoogLeNet son katman
Global Avg Pool	Tüm özellik haritasının ort.	$1 \times 1 \times C$	FC yerine, parametre az
Global Max Pool	Tüm özellik haritasının maks.	$1 \times 1 \times C$	NLP CNN modelleri

Global Average Pooling (GAP): Son özellik haritasını tek bir vektöre düşürür. Bu sayede FC katmana gerek kalmaz — parametre sayısı önemli ölçüde azalır. ResNet, GoogLeNet ve MobileNet'in son katmanı GAP kullanır.

4. Klasik CNN Mimarileri

Model	Yıl	Kat.	Temel Yenilik	ImageNet Top-5
LeNet-5	1998	7	İlk başarılı CNN, el yazısı tanıma	MNIST ~99%
AlexNet	2012	8	GPU eğitimi, ReLU, Dropout, veri artırma	15.3%
VGG-16/19	2014	19	Sadece 3×3 filtre, derinlik artışı	7.3%
GoogLeNet	2014	22	Inception modülü, 1×1 konvolüsyon	6.7%
ResNet-50	2015	50	Skip connection, 152 katman mümkün kıldı	3.57%
DenseNet	2017	121	Tüm önceki katmanlarla bağlantı	3.46%
MobileNet	2017	28	Depthwise conv, mobil cihazlar	4.2% (V1)
EfficientNet	2019	—	Compound scaling — en verimli	1.8% (B7)

5. ResNet Skip Connection

Problem: 50+ katmanlı ağlar 20 katmanlıdan daha kötü sonuç veriyordu. Bu counter-intuitive durum 'degradation problemi' olarak adlandırıldı. Vanishing gradient veya optimizasyon zorluğu nedeniyle derin ağlar düzgün eğitilemiyor, sığ ağları bile taklit edemiyordu.

Çözüm: Residual Learning

Normal blok: $x \rightarrow [\text{Conv} \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{Conv} \rightarrow \text{BN}] \rightarrow F(x) \rightarrow \text{ReLU}$

ResNet blok: $x \rightarrow [\text{Conv} \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{Conv} \rightarrow \text{BN}] \rightarrow F(x) + x \rightarrow \text{ReLU}$

↑

Skip Connection

- Öğrenilecek: $F(x) = H(x) - x$ (rezidüel = hedeften fark)
- $F(x)=0$ ise $H(x)=x$: kimlik dönüşümü — ağ sığ ağı öğrenebilir
- Gradyan doğrudan skip connection üzerinden akar $\rightarrow$ vanishing yok
- ResNet-152: 152 katman — skip connection olmadan eğitilemez

6. Grad-CAM

Gradient-weighted Class Activation Mapping: Modelin karar verirken görüntünün hangi bölgesine odaklandığını görselleştirir. Model yorumlanabilirliği (explainability) için kritik araç.

Adım	İşlem	Formül
1	İleri geçiş yap, hedef sınıf skorunu al	$\hat{y}^c = \text{model}(I)$
2	Son Conv katmana göre gradyan hesapla	$\partial \hat{y}^c / \partial A^k$
3	Her kanal için global ortalama → ağırlık	$\alpha_k = (1/Z) \cdot \sum_{ij} \partial \hat{y}^c / \partial A_{ij}^k$
4	Ağırlıklı feature map toplamı + ReLU	$L = \text{ReLU}(\sum_k \alpha_k \cdot A^k)$
5	Giriş görüntüsü boyutuna ölçekle ve görselleştir	$\text{upsample}(L) + \text{overlay}$

Grad-CAM neden ReLU kullanır? Negatif değerler, hedef sınıf için ZARARLIYSA işaretli bölgeler demektir. Yalnızca pozitif etkili bölgeler görselleştirilir.

7. Framework Kullanımı

TensorFlow / Keras — CIFAR-10 CNN:

Katman	Keras Kodu
Conv2D (same)	layers.Conv2D(32, (3,3), padding='same', activation='relu')
BatchNorm	layers.BatchNormalization()
MaxPool	layers.MaxPooling2D(2, 2)
Dropout	layers.Dropout(0.25)
GlobalAvgPool	layers.GlobalAveragePooling2D()
Dense (çıkış)	layers.Dense(10, activation='softmax')

PyTorch — CIFAR-10 CNN:

Katman	PyTorch Kodu
Conv2d (same)	nn.Conv2d(3, 32, kernel_size=3, padding=1)
BatchNorm2d	nn.BatchNorm2d(32)
MaxPool2d	nn.MaxPool2d(2, 2)
Dropout2d	nn.Dropout2d(0.25)
AdaptiveAvgPool	nn.AdaptiveAvgPool2d((1,1))
Linear (çıkış)	nn.Linear(64, 10)

8. Yaygın Hatalar

Hata 1: Same padding hesabını yanlış yapmak	Same padding için gerekli $P = (K-1)/2$ (K tek sayı olduğunda). $K=3 \rightarrow P=1$, $K=5 \rightarrow P=2$. Stride>1 ile same padding boyutu korumaz.
Hata 2: BatchNorm'u Conv'dan sonra aktivasyondan önce mi sonra mı koymak	Tartışmalı. ResNet orijinal: Conv→BN→ReLU. ResNet v2: BN→ReLU→Conv (pre-activation). Başlangıç için Conv→BN→ReLU standart.
Hata 3: Global Average Pooling sonrası flatten yapmak	GAP çıktısı zaten (batch, channels, 1, 1) — flatten veya squeeze gereklidir PyTorch'ta: <code>x = x.view(x.size(0), -1)</code>
Hata 4: Feature map boyutunu takip etmemek	Her Conv ve Pool katmanı sonrası boyutu hesapla. Hatalı boyut FC katmanda çakışma hatası verir. <code>model.summary()</code> veya <code>print(model)</code> kullan.
Hata 5: Stride=2 ile MaxPool'u birlikte kullanmak	İkisi de boyutu küçültür. Genellikle biri kullanılır. Modern ağlar MaxPool yerine Stride=2 Conv tercih eder (öğrenilmiş boyut azaltma).

9. Özet Tablo

Kavram	Formül / Kural	Amaç
Konvolüsyon	$(I * K)[i,j] = \sum \sum I[i+m,j+n] \cdot K[m,n]$	Yerel özellik çıkarma
Çıkış boyutu	$H_{out} = \text{floor}((H+2P-K)/S)+1$	Boyut hesaplama
Max Pooling	max değer, pencerede	Boyut küçültme, özellik vurgu
Global Avg Pool	tüm HxW ortalaması $\rightarrow 1 \times 1 \times C$	FC yerine, az parametre
1x1 Konv	$K=(1,1)$, kanal değişimi	Boyut azaltma, hesap tasarrufu
Skip Connection	$H(x) = F(x) + x$	Derin ağ eğitimi, vanishing çözüm
Grad-CAM	$L = \text{ReLU}(\sum_k \alpha_k \cdot A^k)$	Model yorumlanabilirliği

10. Kaynaklar

- [1] LeCun, Y. et al. (1998). Gradient-Based Learning Applied to Document Recognition. Proceedings of the IEEE.
- [2] Krizhevsky, A. et al. (2012). ImageNet Classification with Deep CNNs (AlexNet). NeurIPS.
- [3] Simonyan, K. & Zisserman, A. (2014). Very Deep Convolutional Networks (VGG). arXiv:1409.1556.
- [4] He, K. et al. (2016). Deep Residual Learning for Image Recognition (ResNet). CVPR.
- [5] Selvaraju, R. et al. (2017). Grad-CAM: Visual Explanations from Deep Networks. ICCV.
- [6] Tan, M. & Le, Q. (2019). EfficientNet: Rethinking Model Scaling for CNNs. ICML.
- [7] CS231n Stanford — Convolutional Neural Networks. <http://cs231n.github.io>

Sonraki Adım: Modül 07 — Transfer Learning | Feature Extraction · Fine-tuning · VGG16/ResNet50 · Diferansiyel LR